

# A melhor otimização foi a que decidi *não fazer.*

Construí um agente que lê a caixa de apoio ao cliente de uma loja online e escreve rascunhos. A parte difícil não foi ligar um LLM ao email.

453

emails processados em produção

0

enviados sem revisão humana

66%

escalados, quase todos com razão

\$0,008

o teste que travou um deploy

Todos os números verificados contra a base de dados de produção e o código a 3 de setembro de 2026. Onde não existe medição, está escrito que não existe.

# Um LLM resolve isto num playground. Em produção, não.

Uma loja online recebe correspondência o dia inteiro: estado de encomendas, devoluções, defeitos, trocas, reembolsos, queixas. Ligar um modelo à caixa funciona: ele escreve emails de apoio perfeitamente credíveis.

O problema é que a maior parte destas mensagens **não deve** ser respondida automaticamente. Reembolsos movem dinheiro. Alterar uma encomenda exige escrita em sistemas que não devem estar ao alcance de um modelo. E responder com confiança a uma política que não se conhece é pior do que não responder. Um cliente com uma promessa errada dá muito mais trabalho do que um cliente que espera.

## A PERGUNTA DE DESENHO

A pergunta nunca foi como automatizar o apoio ao cliente. Foi *o que é seguro automatizar, e como é que o sistema sabe a diferença.*

# O modelo faz uma coisa só.

Corre de dois em dois minutos. Para cada email decide entre três ações. Tudo o que pode ser decidido em código é decidido **antes** de o modelo ver o caso.

ARQUITETURA



Provar identidade, somar prazos, descartar ruído e validar a decisão vivem em código. Cada uma foi para lá **porque o modelo mostrou, em produção, que a executava mal**. A fronteira foi movida com evidência. Não a desenhei à partida.

# O guardrail é um @property, não uma instrução.

Resolver a encomenda de quem escreve produz quatro níveis de confiança. Só dois libertam os dados para o prompt. O nível intermédio foi **deliberadamente excluído**.

```
class Correspondencia:
    """O resultado de procurar a encomenda de quem escreveu.

    A confiança é decidida aqui, no código, e não pelo modelo: o
    modelo recebe só o resultado. "Adivinhar" uma encomenda é o
    erro mais caro possível deste sistema, porque expõe dados de
    um cliente a outro.
    """

    encomenda: dict | None
    confianca: str # "exata" | "alta" | "media" | "nenhuma"

    @property
    def pode_revelar(self) -> bool:
        """Só se revela ao cliente com identidade provada.

        "media" não chega de propósito: é o nível em que há
        indícios mas não prova, e é exatamente aí que um engano
        mostra a encomenda de outra pessoa.
        """

        return self.encomenda is not None \
            and self.confianca in ("exata", "alta")
```

**Não é uma regra de prompt**

Não há instrução que o modelo possa desobedecer, porque ele nunca vê os dados

**O rascunho vazio é rebaixado**

Se o modelo escolhe “rascunhar” e devolve corpo vazio, o código converte em escalação

**Os prazos são somados em Python**

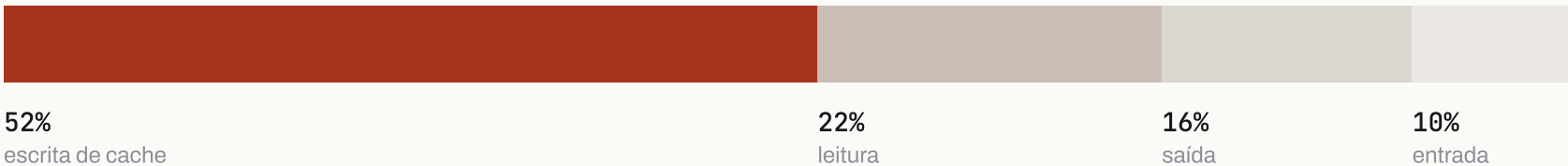
Depois de o modelo ter errado uma soma de 14 dias com a data à frente

Cada uma destas três foi para código **depois** de o modelo demonstrar executá-la mal em produção. Não foi desconfiança de princípio.

# A taxa de cache parecia ótima.

O registo não guardava tokens nenhuns. A única fonte era a fatura mensal, agregada e impossível de atribuir a um email. Antes de otimizar o que quer que fosse, instrumentei: sete colunas novas no registo.

## DECOMPOSIÇÃO DO CUSTO DE UM DIA



89% de acerto de cache parecia resolvido. Mas escrever custa um múltiplo do que custa ler, e por isso *os 11% de falhas custavam 2,4x mais do que todos os acertos somados.*

\$0,0135

ler o prefixo em cache

\$0,248

reescrevê-lo num arranque a frio

MEDIDO • 31 AGO

O problema nunca foi o prompt ser grande. Era o número de vezes que era reescrito.

Estes dois valores são de **31 de agosto**, quando o prefixo tinha duas entradas de cache e o modelo ainda estava em preço de lançamento. Hoje é uma entrada de 35 778 tokens, e ao preço atual custa **\$0,0107 para ler** e **\$0,2147 para reescrever**: a mesma razão de 20 para 1. Ver a página seguinte.

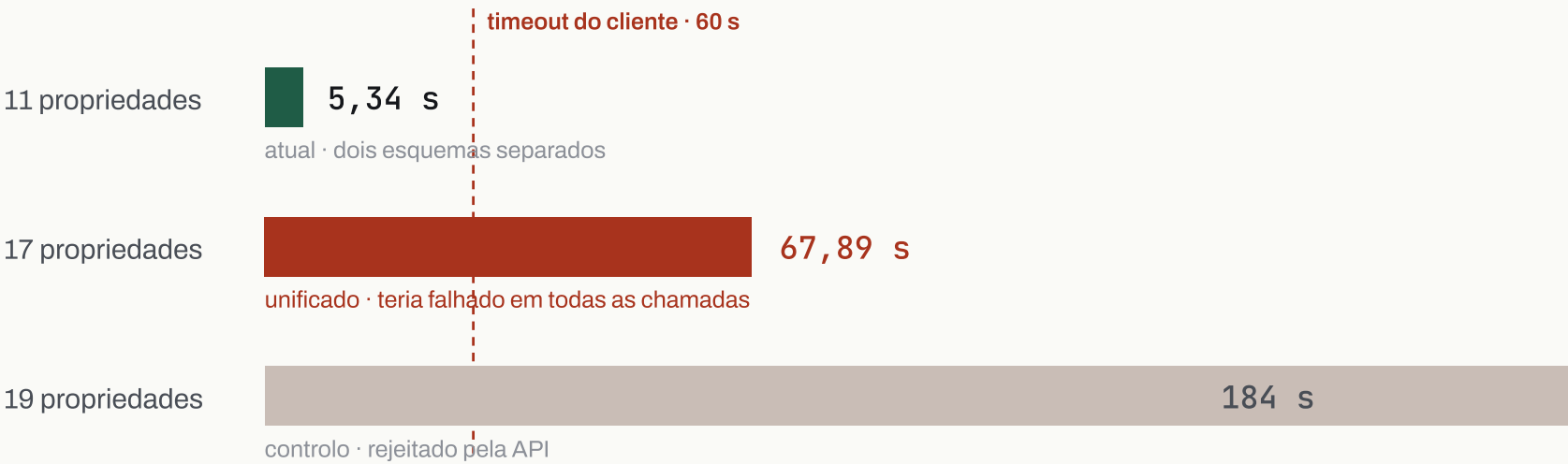
# Unificar parecia obviamente melhor.

A investigação revelou algo que eu não sabia: **havia duas entradas de cache, não uma**. O esquema de saída faz parte do prefixo, por isso as duas chamadas nunca a partilhavam. Num arranque a frio, uma escalação reescrevia o prompt inteiro duas vezes.

A hipótese era evidente: um esquema só, uma entrada só, metade da escrita desaparecia.

## O TESTE: UMA CHAMADA REAL POR CONFIGURAÇÃO

A DECISÃO



DECISÃO · REJEITADA

Teria estourado o timeout em **todas** as chamadas. Falha total, não intermitente.

CUSTO NÃO LINEAR

11 propriedades dão 5 s; 17 dão 68 s. Aparar um campo ou dois também não salvava.

CUSTO DO TESTE

\$0,008

Uma otimização rejeitada depois de um teste *é um bom resultado de engenharia.*

Epílogo: as duas entradas de cache **deixaram de existir a 1 de setembro**. Não por unificar os esquemas, mas por remover a segunda chamada por completo. O mesmo problema, resolvido pela via que a medição não tinha proibido.

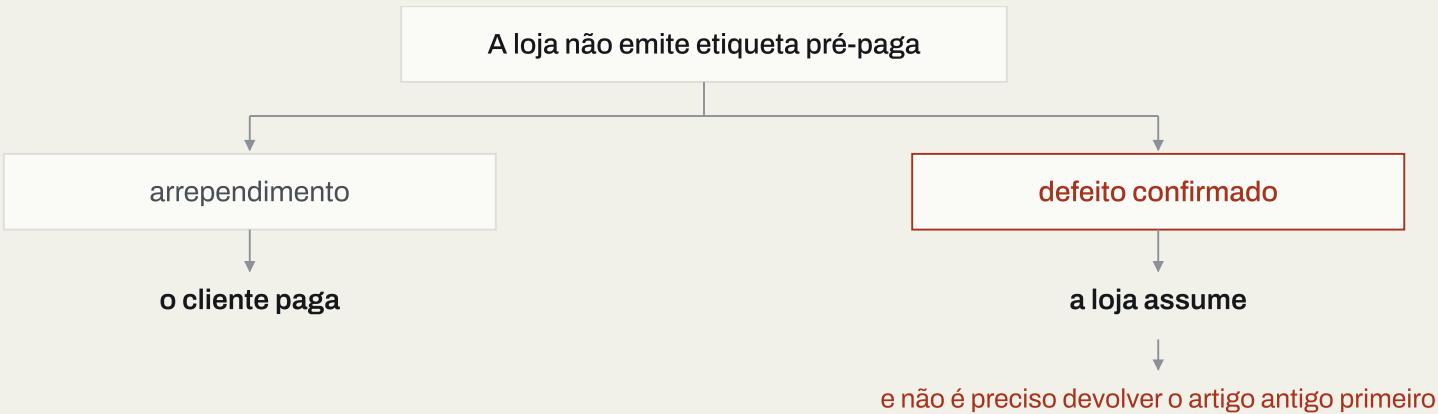
# Uma pergunta simples não é uma regra simples.

A base de conhecimento tem 1 088 linhas em 7 ficheiros e é a única fonte que o modelo pode citar. Escrevê-la foi a parte do projeto que mais tempo me consumiu, e a que menos se parece com programação.

CONHECIMENTO

## PERGUNTA DO CLIENTE

“Quem paga o envio da devolução?”



Duas regras verdadeiras sobre o mesmo assunto, que se contradizem lidas fora de contexto. Tiveram de ficar explicitamente ligadas uma à outra, cada uma a apontar para a outra, para o modelo não aplicar a errada com confiança.

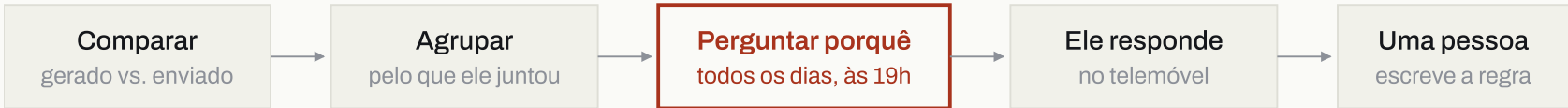
Uma base ambígua não produz respostas erradas. *Produz escalações a mais*, porque o agente é instruído a escalar na dúvida.

O custo da ambiguidade aparece na fatura e na carga de trabalho, não na qualidade.



# Cada edição do lojista é um requisito por escrever.

Quando ele muda um rascunho antes de o enviar, está a corrigir uma regra que eu não escrevi. Isso ficava perdido na caixa dele. Agora não.



Diferenças acima de 90% são cosméticas e ignoram-se. Abaixo de 60% ele reescreveu, e mostra-se o texto todo. Agrupa-se pelo texto que ele acrescentou, não pelo email: o mesmo padrão em clientes diferentes junta-se.

Pergunta o *porquê*, nunca propõe a resposta.  
Uma regra escrita a partir de uma concordância educada é pior do que nenhuma regra.

## A RONDA DE 3 DE SETEMBRO: TRÊS DESFECHOS DIFERENTES

|                     |                                                                                                                                                                    |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Prazo de uma capa   | Ele sabia que já estava resolvido. O assistente não podia saber. Nada entrou na base. A resposta estava certa.                                                     |
| Troca de uns fones  | A regra existia, mas só cobria “o cliente aceita”, e ele tinha perguntado “como é que troco”. Gatilho alargado.                                                    |
| Código de encomenda | A loja tinha mandado o código num print, seis minutos antes. O modelo lê texto: a imagem era invisível. Não era conhecimento em falta. Era um defeito de ingestão. |



# 66% escalavam. Quis baixar o número.

Cada escalação custa mais do que um rascunho. A leitura fácil é “há aqui muito para automatizar”. Passei um dia a ler os motivos, um a um.

| CATEGORIA              | N   | PORQUE ESCALA                                   |
|------------------------|-----|-------------------------------------------------|
| Ação sobre encomenda   | 103 | exige <b>escrita</b> na plataforma de comércio  |
| Compromisso anterior   | 95  | estado que não existe em nenhum sistema legível |
| Julgamento humano      | 52  | garantia, litígio, exceção, gesto comercial     |
| Outras seis categorias | 47  | contexto, identidade, lacuna, inventário        |

■ MEDIDO

Testei a hipótese mais promissora: “as perguntas de reembolso podiam ser respondidas com dados que a plataforma já tem”. Fui buscar as encomendas envolvidas.

2/9

tinham o reembolso registrado. Nas outras 7 estava mesmo por processar

## E LI OS MOTIVOS DAS SEIS CATEGORIAS MAIS PEQUENAS, UM A UM

Não estavam todos certos. **3 diziam que uma mensagem da loja tinha chegado cortada**. Não era julgamento, era um defeito meu: o histórico vinha truncado a 255 caracteres. Corrigido. Outros **5 eram lacunas da base**, e duas fecharam-se esta semana. Os restantes escalam por desenho.

Baixar aquela percentagem exigia dar ao agente escrita na plataforma, *o que apaga o guardrail que torna o sistema tolerável*.

# Um guardrail de prompt contorna-se. Um de infraestrutura não.

22

guardrails documentados, em quatro níveis

MAIS FRACO

Prompt 10

2 Esquema

Código 7

3 Infraestrutura

MAIS FORTE

## Sem Mail.Send

A aplicação nunca pediu a permissão de envio

## Sem escrita na loja

Apenas leitura de encomendas

## Revisão obrigatória

Todo o output passa por uma pessoa

Mesmo que os vinte guardrails anteriores falhem em simultâneo, o pior resultado possível é *um rascunho errado que uma pessoa lê e apaga.*

Nenhum email sai. Nenhuma encomenda muda.

# Sete dos vinte e dois existem porque algo correu mal.

O QUE ACONTECEU

O QUE MUDOU

**O modelo errou o cálculo de um prazo**  
com a data de entrega correta no contexto

O cálculo passou para Python e chega ao modelo já pronto

**Formulários do site descartados**  
pedidos reais apanhados como ruído automático

Exceções explícitas que confirmam o conteúdo antes de descartar

**Uma paragem total sem alerta nenhum**  
a conta ficou sem saldo; o sistema saía com código de sucesso, indistinguível de um dia sem emails

Contagem de passagens seguidas sem decisão.  
**Deteta em ~6 minutos** o que antes podia correr horas

**O deploy disse “Enviado” sem enviar**  
o código de saída não apanhou a falha

O script confirma no servidor que o conteúdo chegou

**Publicar uma regra custa dinheiro**  
a base faz parte do prefixo em cache; cada publicação invalida-o

O deploy compara uma impressão digital do prompt e avisa se aquela publicação é gratuita. Informativo, nunca bloqueante

## O RAIO DE DANO DE TODOS ELES FOI ATRASO, NÃO PERDA

O cursor da caixa só avança dentro do registo de uma decisão. Uma chamada que falha não grava nada, e o email volta a ser lido na passagem seguinte. Nenhuma das avarias acima perdeu um email. Atrasou-o. É a propriedade que torna estes incidentes recuperáveis em vez de custosos.

O padrão comum: nenhum foi encontrado a ler código. Foram todos encontrados a comparar o que o sistema produziu com o que aconteceu a seguir. O sinal de maturidade não é ter guardrails, é a proveniência deles.

INCIDENTES

# Uma diz 97% de rejeição. A outra diz 75% de aceitação.

Meço a qualidade de duas maneiras. Durante algum tempo julguei que uma delas estava avariada.

| PELO ID DO RASCUNHO                | PELO TEXTO ENVIADO                   |
|------------------------------------|--------------------------------------|
| 97%                                | 75%                                  |
| consta como “apagado” (184 de 190) | saiu idêntico ao gerado (137 de 182) |

Ambas estão certas. “Apagado” significa que o objeto do rascunho já não **existe**, não que tenha sido rejeitado: quem responde copia o texto, envia a partir da sua própria janela, e o rascunho desaparece. A medição por id conta objetos; a medição por texto conta trabalho aproveitado. Só a segunda responde à pergunta que interessa.

## A DISTRIBUIÇÃO QUE A MEDIANA ESCONDIA

|                 |  |     |
|-----------------|--|-----|
| exatamente 100% |  | 137 |
| 90 a 99%        |  | 9   |
| 50 a 89%        |  | 20  |
| abaixo de 50%   |  | 16  |

A mediana é 100% e a média é 90%. Nenhuma das duas descreve isto: *três quartos saem intactos, e um em cada oito é reescrito de raiz.*

Publicar só a mediana seria escolher o número a dedo. Os 16 de baixo são onde está o trabalho por fazer. Cada um vira regra nova ou caso de ensaio.

MEDICÃO

O QUE ISTO CUSTOU

O custo caiu 63%.  
A capacidade subiu.  
Não foi de graça.

Dois dias antes das alterações contra dois dias depois. Mesmo sistema, mesma caixa, mesmo tipo de correspondência.

|                                                                                       | 30-31 AGO | 2-3 SET  |          |
|---------------------------------------------------------------------------------------|-----------|----------|----------|
| Tokens pagos por email                                                                | 19 998    | 7 443    | -63%     |
| Custo por email<br>a fatura desceu menos porque o preço do modelo subiu 50% pelo meio | \$0,0475  | \$0,0270 | -43%     |
| Chamadas ao modelo por email                                                          | 1,73      | 1,00     | -42%     |
| Emails resolvidos com rascunho                                                        | 23%       | 40%      | +17 p.p. |
| Rascunhos enviados sem uma palavra mudada                                             | 79%       | 74%      | -5 p.p.  |
| Rascunhos reescritos de raiz                                                          | 6%        | 10%      | +4 p.p.  |

As duas últimas linhas são o preço. E a explicação mais provável está na quarta: o sistema passou a **escrever rascunhos em casos que antes escalava**. Atacar casos mais difíceis faz descer a percentagem que sai intacta. Cinco pontos de aproveitamento por dezassete pontos de casos resolvidos parece-me boa troca, mas é uma troca.

O QUE NÃO POSSO AFIRMAR

**Isto não prova causalidade.** Na mesma janela entraram três alterações: a remoção da segunda chamada, a correção do histórico do fio (vinha truncado a 255 caracteres, e o modelo lia a conversa por uma frincha) e cinco regras novas na base. Qualquer uma delas explica parte do movimento.

Sei *o quê* com números de produção. *Porquê*, só por atribuição. Para o separar era preciso ter mudado uma coisa de cada vez, e não foi o que fiz.

# Os números, e o que eles não provam.

| VOLUME                                       | CUSTO                                          | SEGURANÇA                        | TESTES                         |
|----------------------------------------------|------------------------------------------------|----------------------------------|--------------------------------|
| 453<br>emails processados                    | \$0,0270<br>por email, na arquitetura atual    | 0<br>emails enviados sem revisão | 307<br>testes unitários        |
| 26·66·9<br>% rascunho · escalação · descarte | 96%<br>do prefixo lido da cache                | 0<br>alterações a encomendas     | 98<br>casos no banco de ensaio |
|                                              | 7,3s<br>mediana; 11,8 s no p95, em 22 amostras |                                  |                                |

## QUALIDADE: O GERADO CONTRA O ENVIADO

|                          |                                   |                                                                           |
|--------------------------|-----------------------------------|---------------------------------------------------------------------------|
| 182<br>casos comparáveis | 137<br>saíram idênticos ao gerado | 16<br>foram reescritos de raiz. Cada um vira regra nova ou caso de ensaio |
|--------------------------|-----------------------------------|---------------------------------------------------------------------------|

### A RESSALVA FAZ PARTE DO RESULTADO

São 453 emails, uma caixa, quatro semanas, e a telemetria é mais recente do que o sistema (cobre 252 deles). Escala de loja pequena e sem linha de base limpa: o que generaliza é o método, não os valores absolutos. Com dez vezes o volume, a aritmética da cache muda de sinal.

A latência tem 22 amostras, e isso é pouco. Media-se o custo em dinheiro e não em tempo até ao dia em que escrevi isto. Mediana de 7,3 s e p95 de 11,8 s são o primeiro dia de dados, não uma linha de base.

Nada verifica contradições na base, e a saliência continua por resolver: regras corretas que o modelo por vezes não aplica.

RESULTADOS

Uma média pode esconder uma assimetria de preço.

89% de acerto parecia resolvido, e os 11% restantes custavam mais do que todos os acertos juntos. Só se vê depois de instrumentar: as duas maiores otimizações foram invisíveis até haver colunas de custo no registo.

Uma hipótese rejeitada por medição é um resultado.

Medir custou \$0,008. Reverter aquele deploy teria custado o atendimento de um dia.

Nem toda a automação por fazer é dívida.

Ler os motivos de escalação um a um mostrou que a maior parte estava certa. E escrever as regras da loja, com as suas exceções e prazos de validade, é engenharia por direito próprio.

A métrica certa não era quantos casos automatizei. Era *quantos consegui automatizar com segurança.*

STACK

Python 3.14 · SQLite · systemd · Microsoft Graph ·  
Shopify Admin API (leitura) · Anthropic Messages API ·  
unittest

DELIBERADAMENTE  
AUSENTES

Sem framework web, ORM, fila de mensagens, contentores ou framework de agentes. É um sistema que **uma pessoa sozinha** tem de conseguir manter.

Onde isto parte: o processamento é sequencial e o SQLite é local, por isso uma segunda caixa ou um pico acima de 25 emails por passagem obrigam a repensá-lo. Ainda não aconteceu.

Loja, domínios, endereços, identificadores de encomenda e nomes de pessoas foram omitidos em todo o documento.